

오늘의 작업기록 — 2026년 7월 5일

mini-ledger — 원자성 버그 수정 + 복식부기 = Layer 1 완성

(하루에 셋: “정산 완결”의 반쪽을 메우고, 복식부기 합=0을 만들고, 그걸 실제로 불러 원자성을 증명한 날)

한눈에

항목	내용
목표	정산 원자성 버그 수정 + 복식부기(합=0 검증) → Layer 1 닫기
결과	셋 다 완료 ✓ — Layer 1 전부 완성
커밋	04de86d (정산 검증) → c7794bd (원자성 수정) → 96f828d (복식부기)
오늘의 핵심	★ “검증은 상태 변경 전에”(원자성) + 합=0(복식부기) + 만들고 실제로 호출
새 개념	Map vs Set · getOrDefault · = vs : · += · value vs values() · 필드 vs 지역변수 · e.getMessage
방식	솔로 런 — 트레이스 · 수정 · 복식부기 코드 전부 직접, 문법 다수 교정
의미	mini-ledger Layer 1 완성 (6/19 시작 → 오늘 마지막 조각)

어제 “정산 완결”이라 적었지만 반쪽이었다. 오늘 다른 Claude가 그 구멍(실패 경로 미검증)을 짚었고, 나는 그 지적이 맞음을 확인했다. a=1010을 손으로 트레이스해 “이체는 됐는데 수수료가 안 된 반쪽 커밋 + 먹등성 깨짐”을 눈으로 봤고, “검증을 상태 변경 앞으로” 옮겨 고쳤다. 이어서 복식부기(합=0)를 만들고 — 만들기만 하면 장식이니 — transfer에서 실제로 불러 원자성이 돈 보존을 지키는지 수학으로 증명했다. Layer 1의 마지막 조각이 박혔다.

0. 출발점

- 어제(7/4): 정산 수수료 로직 작성 + 크럭스(2/100=0). 단 Main 검증 못 하고 WIP.
- 오늘 목표: ①정산 완결(Main 검증) ②원자성 버그 ③복식부기 → Layer 1 닫기.
- 다음: Layer 1 완성 → 8월부터 Layer 0(WAL) 위에 진짜 Layer 1 재구축.

PART 1 — 조각 1: 정산 해피패스 검증 (어제 못 한 것)

Main에 platform 계좌 생성 + 이체 + 세 잔액 출력을 직접 짜서 돈 보존 확인.

이체 전: a=2100, b=3500, platform=0 (합 5600)
이체 후: a=1080, b=4500, platform=20 (합 5600) ← 보존 ✓

→ a가 1020 냄(이체 1000 + 수수료 20), b가 1000, platform 20. 합 불변. 커밋 04de86d (WIP 딱지 땀).

★ 함정 하나: transfer가 getAccount("platform") 을 부르니, platform을 이체 전에 안 만들면 AccountNotFoundException. createAccount("platform"...) 이 먼저. (순서 틀리면 예외로 알게 된다.)

PART 2 — ★ 조각 2: 원자성 버그 (오늘의 진짜 배움)

2-1. “정산 완결”은 반쪽이었다

어제 해피패스(돈 충분)만 봤다. 실패 경로를 안 봤다. 다른 Claude가 이 구멍을 정확히 짚었고, 확인해보니 맞았다.

버그: withdraw(amount)가 이미 상태를 바꾼 뒤, withdraw(fee)가 검증 없이 실행된다. fee에서 터지면 이체는 남고 수수료는 안 남는 반쪽.

2-2. a=1010 트레이스 (손으로 → 실행으로 확인)

a=1010으로 transfer("a","b",1000):

- a.withdraw(1000) → a=10
- b.deposit(1000) → b=4500
- TRANSFER 기록
- fee=20
- a.withdraw(20) → a=10인데 20?? → ✖ InsufficientBalance, 여기서 죽음
- ⑨~⑫ 실행 안 됨 → FEE 기록 X, usedKeys.add X

실행 결과(고치기 전): Exception ... Bank.transfer(Bank.java:75) , exit code 1. 죽는 순간 메모리 = a:10, b:4500, platform:0, history:TRANSFER만, usedKeys: 빈.

2-3. 세 재앙 (Q1·Q2·Q3)

- **Q1 반쪽 커밋**: a는 나가고 b는 받았는데 수수료·FEE기록 증발. 돈 보존 깨짐. 온전한 원장 아님.
 - **Q2 원칙 위반**: “상태 바꾸기 전에 검증”(1일차·3증분). 수수료 검증이 withdraw 다음에 있어 이미 바꿔놓고 터짐.
 - **Q3 역등성 깨짐**: `usedKeys.add` (⑫)에 못 닿음 → `usedKeys` 빈 채 → 같은 키 재시도 시 ① 검사 통과 → **이체가 또 실행**. (도장 못 찍어서 재입장 허용되는 꼴.)
- 셋 다 “⑧에서 터져 뒤(⑨~⑫)를 못 마무리”해서 생긴 원자성 문제.

2-4. 수정 — 검증을 앞으로

핵심: 돈 건드리기 전에 “a가 amount+fee 전부 감당되나?” 검증. 못 하면 즉시 throw.

```
long fee = amount * feeRate / 100;           // fee 먼저 계산 (위로 이동!)
if (fromAccount.getBalance() < amount + fee) { // * 검증 (돈 건드리기 전)
    throw new InsufficientBalanceException("잔액이 충분하지 않습니다");
}
fromAccount.withdraw(amount);                 // 통과해야 여기 올
...

```

→ 어제는 fee 계산이 이체 아래 있었다. 검증이 fee를 알아야 하니 위로 이동 = “필요가 위치를 정한다.” 그리고 **amount만이 아니라 amount+fee**를 봐야 한다(둘 다 내야 하니까).

2-5. 검증 (두 경로)

상황	결과
a=2100 (충분)	합 5600 보존, 정상 (안 깨짐) ✓
a=1010 (수수료 못 냄)	a=1010, b=3500 그대로 + history 비어있음 + 예외 잡힘 ✓

→ a=1010에서 이체 후에도 잔액 불변 + 거래내역 빈 = 아무것도 안 바뀜. 반쪽 없음. 잡았다: 잔액이 충분하지 않습니다 . 커밋 c7794bd .

2-6. 씨앗 — 왜 WAL이 필요한가 (예고편)

이렇게 “먼저 검증”으로 고쳐도 한계: 두 withdraw 사이에 프로그램이 죽으면 여전히 반쪽. 진짜 all-or-nothing(중간에 죽어도 롤백)은 트랜잭션이 필요하고, 그게 **Layer 0**의 WAL(8월 최종량). 오늘 이 버그가 “왜 WAL이 필요한가”의 예고편.

PART 3 — 조각 3: 복식부기 (Layer 1 마지막)

3-1. 개념 — 합=0이 돈 보존의 증명

복식부기: 모든 돈 이동은 양쪽에 기록되고, 다 더하면 항상 0. 돈이 나가면(-) 반드시 어딘가로 들어간다(+). 합=0 = 돈이 안 새고 안 생겼다는 수학적 증명. (어제 “합 5600 보존”이 온랩프였다.)

스코프(A안): 기존 잔액(직접 방식) 유지 + “합=0 검증”만 추가. 잔액을 분개 합으로 바꾸는 전면 전환(B)은 안 함 — 그건 8월 WAL 위에서 진짜 Layer 1로. (지금 B로 가면 곧 또 갈아엎음 = 두 번 부수기. 순서·절제.)

3-2. 크릭스 — 거래를 계좌별 부호로 변환

Transaction은 from/to/amount라 부호가 없다. 이걸 “계좌별 +/-“로:

```
TRANSFER a→b 1000 → a:-1000, b:+1000
FEE a→platform 20 → a:-20, platform:+20
REFUND b→a 1000 → b:-1000, a:+1000

```

→ 규칙 통일: from이면 -amount, to면 +amount. 종류 무관, from/to만 보면 됨. → null 처리: DEPOSIT(from=null)·WITHDRAW(to=null)은 null 쪽 건너뛸 (!= null 체크). null 계좌에 부호 매기면 터지니까.

3-3. checkBalance 코드

```

public void checkBalance() {
    Map<String, Long> changes = new HashMap<>(); // 계좌→변동액 (지역변수)
    for (Transaction t : history) {
        if (t.getFrom() != null) {
            changes.put(t.getFrom(), changes.getOrDefault(t.getFrom(), 0L) - t.getAmount());
        }
        if (t.getTo() != null) {
            changes.put(t.getTo(), changes.getOrDefault(t.getTo(), 0L) + t.getAmount());
        }
    }
    long sum = 0;
    for (long value : changes.values()) { // 값들 다 더하기
        sum += value;
    }
    System.out.println("변동 합계: " + sum);
    if (sum == 0) System.out.println("돈 보존됨 (합 = 0)");
    else System.out.println("깨짐! 돈이 새거나 생김");
}

```

- 첫 for = 계좌별 변동 **누적**(from -, to +) / 둘째 for = 그 값들 **합산**
- 자료구조 = **Map**(계좌→변동액). Set 아님 — 값(변동액)이 떨어져서. (6/28 “키+값이면 Map”).

3-4. ★ 만들고 → 실제로 호출 (오늘 완성도의 핵심)

검증 메서드는 **부르지 않으면 장식**. transfer 끝(모든 history.add 다음)에서 호출:

```

usedKeys.add(idempotencyKey);
checkBalance(); // ← 이체+수수료 후 합=0 자동 검증
}

```

→ checkBalance는 history를 읽으니, **검증 대상이 history에 다 기록된 뒤**(맨 끝)에 불러야 한다. (add 전에 부르면 그 거래가 아직 history에 없어 무의미.)

3-5. 검증 — 실행

```

변동 합계: 0
돈 보존됨 (합 = 0)

```

계산: a:-1020(이체-1000, 수수료-20), b:+1000, platform:+20 → 합 0. **a가 낸 것 = b+platform이 받은 것**. 아침에 고친 원자성이 진짜 돈 보존을 지킨다는 수학적 증명. 커밋 **96f828d**.

PART 4 — 오늘 잡은 개념들 (복기용)

- **= vs ::** =값 담기(대입, 아무 데나). ::=목록에서 하나씩 꺼내기(**항상된 for 전용**). 규칙: for문의 “하나씩”만 ::, 나머지 전부 =.
- **Map vs Set**: Set=키만(있나없나). Map=키+값. 계좌마다 변동액(값) 필요 → Map.
- **getOrDefault**: Java가 Map에 내장한 메서드(네가 정의 X). “있으면 그 값, 없으면 기본값(0L)”. put/get/add도 다 Java가 준 것.
- **필드 vs 지역변수**: 필드(메서드 밖)=영구. 지역변수(메서드 안, changes 등)=끝나면 사라짐. changes는 그때그때 계산하고 버려서 지역변수.
- **value vs values()**: **values()** =Map의 값들 전체(목록, 메서드). **value** =거기서 꺼낸 하나(내가 지은 이름). history에서 t 꺼내듯.
- **+= ::** **sum += value = sum = sum + value**. 누적. (++ 는 딱 1 증가하는 특수판.)
- **0L**: Long 타입 0. 그냥 0 은 int 취급이라, Map 값이 Long이면 기본값도 Long(0L)으로 맞춤.
- **!= null**: null이 아니면. !=아니다(not), != =같지 않다.
- **e / e.getMessage()**: catch가 잡은 예외를 e 로(타입 뒤 이름). **e.getMessage()** =그 예외에서 메시지 꺼내기(게터 원리). 메시지는 Bank throw에 한 번 쓰고 재사용(동기화).
- **매직 넘버 → 필드**: 코드에 박힌 숫자는 흠어지면 못 바꿈. feeRate 필드로 = 한 곳에서 관리·동기화.
- **id 이름 관례(JPA)**: 자기 식별자=id(getId), 남을 참조+=Id(txId). Transaction의 계좌 참조는 from/to. Account 자기 id는 getId.

5. 산출물

- 커밋 3개: **04de86d** (정산 Main 검증) → **c7794bd** (원자성 수정) → **96f828d** (복식부기 합=0).
- **Bank.java**: transfer에 amount+fee 사전 검증 + checkBalance 메서드 + transfer 끝에서 호출.
- 커밋 계보: **2de4b3a** (정산 WIP) → **04de86d** → **c7794bd** → **96f828d** (복식부기)

6. Layer 1 완성 현황

감사로그	✓ 6/28	역등성	✓ 6/29
조회	✓ 6/30·7/1	환불	✓ 7/2·7/3
정산	✓ 7/4·7/5	복식부기	✓ 7/5 ← 오늘

mini-ledger Layer 1 전부 완성. (학습판. 진짜 Layer 1은 8월 WAL 위에 재구축.)

7. 다음 작업

- (선택) 통합 테스트 — 역등성+환불+정산 테스트를 지우지 말고 쌓기 (7/5에 나온 “회귀 테스트는 쌓는다”).
- (선택) 코드 정리 — id/txId 이름 통일, gitignore(out/ .idea/ *.iml).
- 8월: Layer 0 (WAL) — 오늘 원자성 버그가 예고한 그것. 최종량(가중치 6).

8. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. 원자성 버그에서, 검증을 withdraw “앞”으로 옮겨야 하는 이유는? “뒤”에 있으면 뭐가 남나?
2. 검증 조건이 amount 가 아니라 amount + fee 인 이유는?
3. “먼저 검증”으로 고쳐도 남는 한계는? 진짜 all-or-nothing엔 뭐가 필요하나?
4. 복식부기에서 “합=0”이 증명하는 것은? DEPOSIT/WITHDRAW가 섞이면 왜 0이 안 되나?
5. 거래를 분개로 바꿀 때 from/to의 부호 규칙은? null은 왜 건너뛰나?
6. checkBalance에서 changes를 Set이 아니라 Map으로 한 이유는?
7. checkBalance를 transfer “맨 끝”에서 부르는 이유는? history.add 전에 부르면?
8. changes.getDefault(키, 0L) 에서 0L의 역할은? 그냥 0이면?
9. sum += value 와 for (long value : changes.values()) 에서 += 와 : 은 각각 무슨 뜻인가?

8.5 트러블슈팅 — 잔디가 안 깎이던 문제 (커밋 이메일)

Layer 1 완성 + 대시보드 push까지 다 했는데, GitHub 잔디(contribution 그래프)가 안 깎였다. push는 분명 성공했는데(원격 origin/main이 최신 커밋을 가리킴) 초록 네 모가 안 났다. 파고들었더니 인프라 문제였고, 추적 끝에 원인을 잡았다.

증상 vs 사실

- push는 성공했다: git log 에서 origin/main 이 최신 커밋을 가리키고 up to date . 즉 커밋은 GitHub에 확실히 올라갔다.
- 근데 잔디는 안 깎였다: push(커밋이 올라감)와 잔디(그 커밋이 "내 것"으로 인정됨)는 별개다.

원인 추적 (단계별)

1. 커밋 이메일 확인 — 두 repo의 커밋 author 이메일이 서로 달랐다: - mini-ledger 로컬: won77544@gmail.com - backend-journey : won7754@naver.com (VPS 글로벌 기본값) -> 둘 다 내가 의도적으로 넣은 적 없는 이메일. (mini-ledger 세팅 중 로컬에 박힌 걸로 추정.)
2. GitHub 계정 이메일 확인 — Settings → Emails: 등록된 실제 이메일은 taewoowon@gmail.com 하나뿐.
3. ★ 결정적 단서 — 계정에 이메일 프라이버시(private email)가 켜져 있었다. 그래서 GitHub은 모든 web-based Git 작업을 noreply 주소로 연결한다: 288912277+Taewoo-Won@users.noreply.github.com

왜 안 깎였나 (핵심)

GitHub이 커밋을 "내 것"으로 인정해 잔디를 깎으려면, 커밋 이메일이 다음 중 하나여야 한다: - 계정에 등록·인증된 이메일 (taewoowon@gmail.com), 또는 - noreply 주소 (288912277+Taewoo-Won@users.noreply.github.com)

→ 근데 내 커밋들은 둘 다 아닌 엉뚱한 이메일(won77544, won7754)로 찍혀 있었다. 그래서 GitHub이 "누구 커밋인지 모름" 처리 → 잔디 안 깎임. (mini-ledger도 사실 계속 안 깎이고 있었는데, "매일 push했으니 깎였겠지"라고 가정만 하고 있었던 것.)

해결

프라이버시를 유지하면서(실제 gmail 노출 안 함) 잔디가 깎이도록, 커밋 이메일을 noreply 주소로 통일:

1. git 설정 3곳 통일 (글로벌 + backend-journey 로컬 + mini-ledger 로컬): git config --global user.email "288912277+Taewoo-Won@users.noreply.github.com" git config --global user.name "Taewoo-Won" # backend-journey, mini-ledger 각 폴더에서도 로컬로 동일 설정 (로컬이 글로벌을 덮으므로) → 앞으로의 모든 커밋은 올바른 이메일로 찍힘 = 잔디 깎임.
2. 이미 찍힌 커밋 소급 반영 — 오늘 대시보드 커밋은 이미 틀린 이메일로 찍혔으니, author를 고쳐서 다시 push: git commit --amend --author="Taewoo-Won <288912277+Taewoo-Won@users.noreply.github.com>" --no-edit git push --force → 커밋 해시가 f8ffab7 → 5eafaf3 로 바뀌며 올바른 이메일로 재기록. (force push는 혼자 쓰는 repo라 안전.)

개념 정리

- push ≠ 잔디: push는 "커밋이 원격에 올라감", 잔디는 "그 커밋이 내 계정 것으로 인정됨". 둘은 별개. 이메일이 안 맞으면 push는 되도 잔디는 안 깎인다.
- 이메일 프라이버시: GitHub 계정이 이메일을 숨기면(private), 커밋은 실제 이메일 또는 숫자+아이디@users.noreply.github.com 으로 찍혀야 인정된다.
- 로컬 vs 글로벌 git config: 폴더별 로컬 설정이 글로벌을 덮는다. 그래서 repo마다 이메일이 다를 수 있고(이번 사달의 원인), 확실히 하려면 글로벌 + 각 로컬을 다 맞춰야 한다.
- --amend + --force : 마지막 커밋의 author/메시지를 고쳐 다시 쓰고, 원격 히스토리를 덮어씀. 혼자 쓰는 repo에서만 안전.

교훈: "push 성공 = 잔디 깎임"이 아니다. 잔디는 커밋 이메일이 계정과 일치해야 깎인다. 그리고 "깎였겠지"라고 가정하지 말고 실제 그래프를 눈으로 확인할 것. (오늘 두 repo 다 몇 주간 잔디가 안 깎이고 있었을 수 있는데, 가정만 하고 있었다.)

9. 회고

오늘은 세 개를 했고, 셋이 하나로 이어졌다.

첫째, **어제 “완결”이 반쪽이었다는 걸 인정한 것**. 해피패스만 보고 “됐다”고 했는데, 다른 Claude가 실패 경로 구멍을 줬었다. 처음엔 “FEE 뜨는데?”라고 반박했지만 — 그건 $a=2100$ (충분)일 때였고, $a=1010$ (부족)으로 바꾸니 반쪽 커밋이 그대로 드러났다. 성급함을 실행으로 확인하고 고친 것. 어제 크릭스가 “ $2/100=0$ ”이었다면, 오늘은 “해피패스만 보면 안 된다”였다.

둘째, **원자성을 손으로 트레이스해 눈으로 본 것**. ⑧에서 터지면 뒤(수수료·키 저장)가 다 날아가 반쪽+역등성 깨짐. 이걸 표로 따라가며 봤고, “검증은 상태 변경 전에”로 고쳤다. 코드 두세 줄이지만, 원장이 왜 깨지는지를 몸으로 안 게 값지다.

셋째, **복식부기 — 그리고 만든 걸 실제로 부른 것**. 거래를 계좌별 부호로 바꿔 합=0인지 보는 checkBalance를 만들고, transfer 끝에서 호출했다. 다른 Claude 말대로 “만들고 안 부르면 장식”이라 — 부르는 것까지가 오늘이었다. 이체할 때마다 합=0이 자동 검증되니, 아침에 고친 원자성이 진짜 돈을 보존한다는 게 매번 수학으로 증명된다.

Map이 처음이라 많이 헤맸다. `=` vs `:`, value vs values(), getOrDefault가 어디서 왔는지, 이중 changes가 뭔지 — 하나씩 물고 늘어졌다. 뿌듯하지만, 이 개념들이 아직 손에 완전히 붙진 않았다는 것도 안다. 복기로 굳혀야 한다. 오늘은 “통과”시켰고, 체화는 다음이다.

그래도 — Layer 1을 닦았다. 6/19에 빈 파일로 시작해서, 감사로그·역등성·조화·환불·정산·복식부기까지. 학습판이지만, 결제 시스템의 뼈대를 처음부터 끝까지 내 손으로 통과시켰다. 8월 WAL 위에 진짜 Layer 1을 지을 때, 오늘 배운 게 복리로 돌아올 것이다.

마지막으로, 예상 못 한 인프라 버그도 하나 잡았다. 다 끝난 줄 알았는데 잔디가 안 깎여서 — push는 됐는데 왜? 하고 파봤더니 커밋 이메일이 계정과 안 맞았던 거다(이메일 프라이어비시 때문에 noreply 주소로 찍혔어야 했다). "push 성공 = 잔디 깎임"이라는 내 가정이 틀렸고, 실제로는 두 repo가 몇 주간 잔디가 안 깎이고 있었을 수 있다. 코드 버그(원자성)를 "가정하지 말고 실행으로 확인"으로 잡은 날, 인프라 버그도 똑같은 방식으로 잡았다 — 가정을 의심하고 실재를 확인하는 것. 우연히도 오늘의 두 버그가 같은 교훈을 가리켰다.

한 줄: “완결”의 반쪽을 인정하고 원자성으로 메운 뒤, 복식부기 합=0을 만들어 실제로 불러 돈 보존을 증명한 날 — mini-ledger Layer 1을 내 손으로 닦았다.